2. Develop a Semantic Search System Using Cosine Similarity

Aim

To develop a semantic search system that retrieves the most relevant documents for a given user query by comparing document embeddings and query embeddings using cosine similarity.

Algorithm

1. Import the required Python libraries for semantic search and similarity calculation.
2. Create a collection of sample documents containing different information.
3. Load a pre-trained Sentence Transformer model.
4. Convert all documents into numerical embeddings.
5. Get a search query from the user.
6. Generate the embedding representation of the query.
7. Calculate cosine similarity between the query embedding and document embeddings.
8. Sort the documents based on their similarity scores in descending order.
9. Display the most relevant documents with their similarity scores.

Code:

```
# Install required library

!pip install -q sentence-transformers scikit-learn

# Import libraries

from sentence_transformers import SentenceTransformer

from sklearn.metrics.pairwise import cosine_similarity

# Create sample documents

documents = [

    "Artificial Intelligence is used to create intelligent machines.",

    "Machine Learning helps computers learn from data.",

    "Python is used for software development.",

    "Deep Learning is based on artificial neural networks.",

    "Cloud Computing provides internet-based services.",

    "Natural Language Processing helps computers understand text."
```

```
]

# Load pre-trained model
model = SentenceTransformer('all-MiniLM-L6-v2')

# Generate document embeddings
document_embeddings = model.encode(documents)

# Get search query
query = input("Enter your search query: ")

# Generate query embedding
query_embedding = model.encode([query])

# Calculate cosine similarity
scores = cosine_similarity(
    query_embedding, document_embeddings
)[0]

# Rank documents
results = sorted(
    zip(documents, scores),
    key=lambda x: x[1],
    reverse=True
)

# Display search results
print("\nSEMANTIC SEARCH RESULTS")
```

```
for i, (doc, score) in enumerate(results, start=1):
    print(f"\nRank {i}")
    print("Document:", doc)
    print("Similarity Score:", round(float(score), 4))
```

Output:

```
Loading weights: 100%|██████████| 103/103 [00:00<00:00, 2960.58it/s]
Enter your search query: How do computers learn from data?

SEMANTIC SEARCH RESULTS

Rank 1
Document: Machine Learning helps computers learn from data.
Similarity Score: 0.6936

Rank 2
Document: Natural Language Processing helps computers understand text.
Similarity Score: 0.4559

Rank 3
Document: Artificial Intelligence is used to create intelligent machines.
Similarity Score: 0.4426

Rank 4
Document: Deep Learning is based on artificial neural networks.
Similarity Score: 0.409

Rank 5
Document: Python is used for software development.
Similarity Score: 0.2022

Rank 6
Document: Cloud Computing provides internet-based services.
Similarity Score: 0.1688
```

Result

Thus, a semantic search system was successfully developed using Sentence Transformer embeddings and cosine similarity to retrieve relevant documents based on user queries.